

AV2 Codec ISO Media File Format Binding

Working Group Draft, 6 October 2026



This version:

<https://aomediacodec.github.io/av2-isobmff/v1.0.0.html>

Issue Tracking:

[GitHub](#)

Editors:

TBD

TBD

Warning

This specification is still at draft stage and should not be referenced other than as a working draft.

Copyright 2026, [Alliance for Open Media](#)

Licensing information is available at <https://www.aomedia.org/license/>

The MATERIALS ARE PROVIDED “AS IS.” The Alliance for Open Media, its members, and its contributors expressly disclaim any warranties (express, implied, or otherwise), including implied warranties of merchantability, non-infringement, fitness for a particular purpose, or title, related to the materials. The entire risk as to implementing or otherwise using the materials is assumed by the implementer and user. IN NO EVENT WILL THE ALLIANCE FOR OPEN MEDIA, ITS MEMBERS, OR CONTRIBUTORS BE LIABLE TO ANY OTHER PARTY FOR LOST PROFITS OR ANY FORM OF INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES OF ANY CHARACTER FROM ANY CAUSES OF ACTION OF ANY KIND WITH RESPECT TO THIS DELIVERABLE OR ITS GOVERNING AGREEMENT, WHETHER BASED ON BREACH OF CONTRACT, TORT (INCLUDING NEGLIGENCE), OR OTHERWISE, AND WHETHER OR NOT THE OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Abstract

This document specifies the storage format for [AV2] bitstreams in [ISOBMFF] tracks as well as in [CMAF] files.

Table of Contents	Pages
1 Terminology	4
2 Bitstream features overview	4
3 Basic Encapsulation Scheme	5
3.1 General Requirements & Brands	6
3.2 AV2 Sample Entry and Configuration	6
3.2.1 AV2 Sample Entry	6
3.2.2 AV2 Codec Configuration Box	7
3.2.2.1 config_obu	7
3.3 AV2 Sample Format	9
3.3.1 General	9
3.3.2 OBUs in samples	10
3.3.3 Sync sample	10
3.3.4 Layers	11
3.3.5 AV2 sample reconstruction	11
4 CMAF AV2 track format	12
5 Common Encryption	12
6 Codecs Parameter String	12
Index	12
Terms defined by this specification	12
Terms defined by reference	13
References	15
Normative References	15

List of Figures

Figure 3.1: List of AV2 OBU types and where they can be found	6
Figure 3.2: Example mapping of AV2 temporal units into the ISOBMFF sample format.	7

§ 1. Terminology

The key words "**MUST**", "**MUST NOT**", "**REQUIRED**", "**SHALL**", "**SHALL NOT**", "**SHOULD**", "**SHOULD NOT**", "**RECOMMENDED**", "**NOT RECOMMENDED**", "**MAY**", and "**OPTIONAL**" in this document are to be interpreted as described in BCP 14 [RFC2119] [RFC8174] when, and only when, they appear in all capitals, as shown here.

All of the text of this specification is normative except sections explicitly marked as non-normative, examples, and notes.

Examples in this specification are introduced with the words "for example" or are set apart from the normative text with `class="example"`, like this:

EXAMPLE 1

This is an example of an informative example.

Informative notes begin with the word "Note" and are set apart from the normative text with `class="note"`, like this:

NOTE: This is an example of an informative note.

§ 2. Bitstream features overview

An [AV2 bitstream](#) is composed of a sequence of [OBUs](#) (Open Bitstream Units), grouped into [Temporal Units](#). A Temporal Unit consists of a [Temporal Delimiter OBU](#) and all following OBUs up to, but not including, the next Temporal Delimiter OBU. Each Temporal Unit is associated with a single presentation time.

Every OBU begins with a 1- or 2-byte header that carries the [obu_type](#) and the temporal layer identifier [obu_tlayer_id](#). When the OBU header extension is present (i.e. when [obu_header_extension_flag](#) is equal to 1 in that header), two additional layer identifiers are carried: the embedded layer identifier [obu_mlayer_id](#) and the extended layer identifier [obu_xlayer_id](#). Depending on its type, an OBU can carry configuration information (e.g., a [Sequence Header OBU](#) or [Layer Configuration Record OBU](#)), metadata, or coded video data.

A Temporal Unit **MAY** contain multiple coded pictures but only one is output. AV2 defines a number of coded-frame OBU types, including [Closed Loop Key OBUs](#), [Open Loop Key OBUs](#), regular and leading tile-group OBUs, [Switch OBUs](#), [OBU_RAS_FRAME](#), [Bridge Frame OBUs](#), and TIP (Temporally Interpolated Prediction) frames.

Frames that can be decoded without reference to other frames are [Key Frames](#) (carried in a [Closed Loop Key OBU](#) or an [Open Loop Key OBU](#)) and [Intra frames](#). All other coded frames - including [Inter Frames](#), [Switch Frames](#), RAS frames, Bridge frames, TIP frames, and frames with [show_existing_frame](#) equal to 1 - have coding dependencies on other frames in the same or previous Temporal Units.

A [Closed Random Access](#) is the random access process that applies to an extended layer whose first coded frame is a [Key Frame](#) carried in a [Closed Loop Key OBU](#). It starts a new coded video sequence for the extended layer; the layer's reference frame buffers are invalidated and every subsequent frame of that layer can be decoded. This is the primary random-access mechanism.

An [Open Random Access](#) is the random access process that applies to an extended layer whose first coded frame is a [Key Frame](#) carried in an [Open Loop Key OBU](#). During sequential decoding, an [Open Random Access](#) point does not start a new coded video sequence; however, when a decoder initiates decoding at that point, the decoding process is treated as if it were the start of a new coded video sequence for the extended layer. [Leading frames](#) that follow an [Open Random Access](#) point **MAY** reference frames that precede it, and are discarded when decoding begins at the [Open Random Access](#) point.

In addition to Key Frames, AV2 provides two further mechanisms that support random access or stream switching: a RAS frame (OBU_RAS_FRAME) is an inter-predicted frame that uses long-term reference frames and enables random access or switching without inserting a full key frame; a [Switch Frame](#) (OBU_SWITCH) is an inter-predicted frame that enables switching between representations at a given point, typically in adaptive streaming scenarios.

AV2 supports three scalability dimensions: temporal layers (identified by [obu_tlayer_id](#)), embedded layers (identified by [obu_mlayer_id](#)), and extended layers (identified by [obu_xlayer_id](#)). The reserved [obu_xlayer_id](#) value GLOBAL_XLAYER_ID (equal to 31, as defined in [\[AV2\]](#)) denotes global scope and is used by OBUs that do not belong to any specific extended layer (such as the [Temporal Delimiter OBU](#)). A bitstream that uses a single distinct non-global value of [obu_xlayer_id](#) is a [Singlestream](#) bitstream; a bitstream that uses two or more distinct non-global values is a [Multistream](#) bitstream.

AV2 defines two distinct [Metadata OBU](#) types for carrying metadata: OBU_METADATA_SHORT, using the [metadata short OBU syntax](#), and OBU_METADATA_GROUP, using the [metadata group OBU syntax](#). A separate OBU type carries film grain synthesis parameters; its presence is indicated by the [film_grain_params_present](#) flag in the [Sequence Header OBU](#).

The [Sequence Header OBU](#) also carries the bitstream's profile, level, and - when applicable - tier, as defined in Annex A of [\[AV2\]](#).

The [Sequence Header OBU](#) carries a [monotonic_output_order_flag](#) that defines the output mode of the associated coded video sequence.

When [monotonic_output_order_flag](#) is equal to 1, the output order of coded output frame equals their decoding order within the coded video sequence. Within a [sample](#), the composition time (CTS) of the associated output frame is equal to the [sample](#)'s decoding time (DTS); the [sample](#) **MAY** additionally contain hidden frames.

When [monotonic_output_order_flag](#) is equal to 0, the output order of coded output frames **MAY** differ from their decoding order. The composition time of each [sample](#) is then derived from a ['ctts](#) box that signals the offset between the [sample](#)'s decoding time (DTS) and the composition time of its associated output frame. As in the monotonic case, a [sample](#) **MAY** additionally contain hidden frames.

NOTE: Non-monotonic output mode is new in AV2; AV1 supports only the monotonic mode.

§ 3. Basic Encapsulation Scheme

This section describes the basic data structures used to signal encapsulation of [AV2 bitstreams](#) in [\[ISOBMFF\]](#) containers.

There are two locations where **OBUs** can be found in a file conformant to this specification:

- Some **High-Level Syntax OBUs**, such as the **Sequence Header OBU**, are carried in the **AV2CodecConfigurationBox** as described in § 3.2 **AV2 Sample Entry and Configuration**. This corresponds to "availability through external means" as described in the [AV2] specification. These **OBUs** apply to all **samples** of the corresponding **AV2SampleEntry**.
- The other **OBUs** are stored in **samples**. This corresponds to "presence in the bitstream" as described in the [AV2] specification.

An **AV2 sample** is the union of a **sample** inclusive of the **OBUs** it carries, the **OBUs** carried in the **AV2CodecConfigurationBox** of the corresponding **AV2SampleEntry**, and an implicit **Temporal Delimiter OBU**.

NOTE: This specification refers to **samples** as defined in [ISOBMFF], which are not to be mistaken with **samples** or **sample values** as defined in the [AV2] specification.

§ 3.1. General Requirements & Brands

A file conformant to this specification satisfies the following:

- It SHALL conform to the normative requirements of [ISOBMFF]
- It SHALL have the **'av02'** brand among the compatible brands array of the **FileTypeBox**
- It SHALL contain at least one track using an **AV2SampleEntry**, possibly transformed by encryption as specified in § 5 **Common Encryption**
- It SHOULD indicate a structural ISOBMFF brand among the compatible brands array of the **FileTypeBox**, such as **'iso6'**
- It SHALL contain at least one **AV2 sample**
- Each **AV2 sample** it contains SHALL form exactly one [AV2] **Temporal Unit** if reconstructed as described in § 3.3.5 **AV2 sample reconstruction**

§ 3.2. AV2 Sample Entry and Configuration

§ 3.2.1. AV2 Sample Entry

The **AV2SampleEntry** extends **VisualSampleEntry** and signals a track carrying **AV2 samples**.

```
class AV2SampleEntry extends VisualSampleEntry('av02') {  
    AV2CodecConfigurationBox config;  
}
```

An **AV2SampleEntry** SHALL contain exactly one **AV2CodecConfigurationBox**. An **AV2SampleEntry** SHALL contain exactly one **'colr'** box with the **'nclx 'colour_type'**. Each value among **colour primaries**, **transfer characteristics**, and **matrix coefficients** carried in that **'colr'** box:

- SHOULD equal the corresponding effective value of the AV2 bitstream carried by the track if the latter is anything but 2 (Unspecified);
- SHALL take precedence over the corresponding effective value of the AV2 bitstream carried by the track.

The value of `full_range_flag` carried in that `'colr'` box SHOULD equal the corresponding effective value of the AV2 bitstream carried by the track and SHALL take precedence over the corresponding effective value of the AV2 bitstream carried by the track. An `AV2SampleEntry` SHALL contain exactly one `'pixi'` box. That box SHALL have the least significant bit of `'px_flags'` enabled. The constraints described in § 3.3.5 [AV2 sample reconstruction](#) apply to that `AV2SampleEntry`.

NOTE: An `AV2SampleEntry` may contain additional sub-boxes permitted by [ISOBMFF] for a `VisualSampleEntry`, such as `'colr'` with a `'colour_type'` value other than `'nclx'`, `'pasp'`, `'clli'`, `'mdcv'` or `'btrt'`. They may be located before and/or after the `AV2CodecConfigurationBox`.

§ 3.2.2. AV2 Codec Configuration Box

The `AV2CodecConfigurationBox` carries the decoder initialisation data required to decode the `AV2 samples` of a track.

```
aligned(8) class AV2CodecConfigurationBox extends Box('av2C')
{
    unsigned int(8) reserved1 = 0;
    unsigned int(8) config_obus_count_minus1;
    for (i = 0; i < config_obus_count_minus1 + 1; ++i) {
        unsigned int(8) config_obu[];
    }
}
```

`reserved` SHALL be set to 0. Readers SHALL ignore its value.

`config_obus_count_minus1` is the number of `config_obu` minus one.

`config_obu` is a byte array carrying one OBU and its length as described in § 3.2.2.1 [config_obu](#).

§ 3.2.2.1. *config_obu*

Each `config_obu` SHALL carry a single OBU preceded by a `leb128()` `num_bytes_in_obu` value giving the byte length of that OBU, as specified in the [Length delimited bitstream format](#) from Annex B of [AV2]. The OBUs within an `AV2CodecConfigurationBox` SHALL appear in the order specified by the [ordering of OBUs in a temporal unit](#) in [AV2], with the following constraint:

- the OBUs within an `AV2CodecConfigurationBox` SHALL NOT contain OBUs of the types forbidden there as stated in [Figure 3.1](#).

AV2 OBU type	Be present in AV2CodecConfigurationBox (in config_obu)	Be present in a sample
OBU_SEQUENCE_HEADER	SHALL	SHALL NOT
OBU_TEMPORAL_DELIMITER	SHALL NOT	SHALL NOT (the AV2 sample boundary already delimits the temporal unit)
OBU_MULTI_FRAME_HEADER	SHALL NOT	MAY
OBU_CLOSED_LOOP_KEY	SHALL NOT	MAY
OBU_OPEN_LOOP_KEY	SHALL NOT	MAY
OBU_LEADING_TILE_GROUP	SHALL NOT	MAY
OBU_REGULAR_TILE_GROUP	SHALL NOT	MAY
OBU_METADATA_SHORT or OBU_METADATA_GROUP, when its contents do not change for the duration of the active AV2SampleEntry	MAY	SHOULD NOT
OBU_METADATA_SHORT or OBU_METADATA_GROUP, otherwise	SHOULD NOT	MAY
OBU_SWITCH	SHALL NOT	MAY
OBU_LEADING_SEF	SHALL NOT	MAY
OBU_REGULAR_SEF	SHALL NOT	MAY
OBU_LEADING_TIP	SHALL NOT	MAY
OBU_REGULAR_TIP	SHALL NOT	MAY
OBU_BUFFER_REMOVAL_TIMING	SHALL NOT	MAY
OBU_LAYER_CONFIGURATION_RECORD	MAY	SHALL NOT
OBU_ATLAS_SEGMENT	SHALL NOT	MAY
OBU_OPERATING_POINT_SET	SHALL NOT	MAY
OBU_BRIDGE_FRAME	SHALL NOT	MAY
OBU_MSDO	SHALL NOT	MAY
OBU_RAS_FRAME	SHALL NOT	MAY
OBU_QUANTIZATION_MATRIX	SHALL NOT	MAY
	MAY	SHOULD NOT

OBU_FILM_GRAIN, when its contents do not change for the duration of the active AV2SampleEntry		
OBU_FILM_GRAIN, otherwise	SHOULD NOT	MAY
OBU_CONTENT_INTERPRETATION	MAY	SHALL NOT
OBU_PADDING	SHALL NOT	SHALL NOT (byte-level padding within a track is achieved using ISOBMFF mechanisms, e.g. ‘free’/‘skip’ boxes or sample size signalling)

Figure 3.1: List of AV2 OBU types and where they can be found

For a [Multistream](#) bitstream, the [Sequence Header OBU](#) of each extended layer together with the stream’s global configuration OBUs SHALL be carried by the associated [AV2CodecConfigurationBox](#), in the order specified by the [ordering of OBUs in a temporal unit](#) in [AV2].

ISSUE 1 Editor’s note: for a [Multistream](#) bitstream, the [AV2CodecConfigurationBox](#) contains a [Sequence Header OBU](#) for each extended layer. The impact on multi-layer use cases — including how a reader determines the active decoder configuration per extended layer from multiple Sequence Header OBUs — needs investigation. Also how profile / level is signalled to decoder API for layers. etc.

§ 3.3. AV2 Sample Format

§ 3.3.1. General

The [AV2 sample](#) boundary delimits the temporal unit.

Each OBU within a [sample](#) SHALL be framed using the [Length delimited bitstream format](#) specified in Annex B of [AV2]: each OBU is preceded by a `leb128() num_bytes_in_obu` field giving the byte length of the OBU that follows. [Figure 3.2](#) illustrates this mapping.

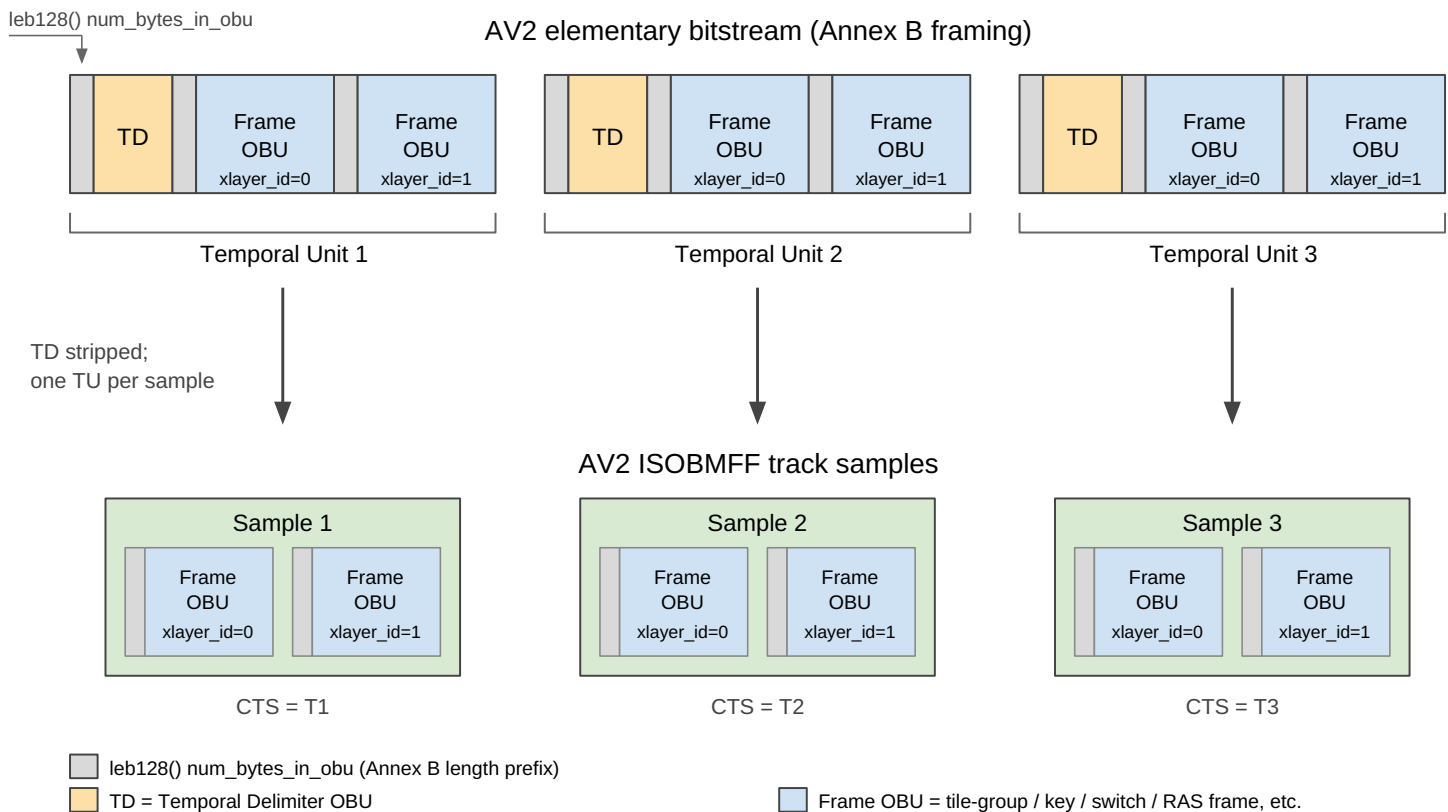


Figure 3.2: Example mapping of AV2 temporal units into the ISO BMFF sample format.

§ 3.3.2. OBUs in samples

The **OBUs** within a **sample** SHALL appear in the order specified by the **ordering of OBUs in a temporal unit** in [AV2], with the following constraint:

- the **OBUs** within a **sample** SHALL NOT contain **OBUs** of the types forbidden there as stated in Figure 3.1.

ISSUE 2 Editor's note: the Content Interpretation OBU (OBU_CONTENT_INTERPRETATION) carries picture-level interpretation metadata (for example, colour description) that overlaps ISO BMFF container-level signalling such as the ColourInformationBox. Whether to carry it in the track or replace it entirely with container-level signalling is to be resolved in this version.

§ 3.3.3. Sync sample

An **AV2 sample** is a **sync AV2 sample** if, for every **coded extended layer unit** present in the **AV2 sample**, the first coded frame OBU of that coded extended layer unit has obu_type equal to OBU_CLOSED_LOOP_KEY. Such an **AV2 sample** is a **Closed Random Access** point for the carried bitstream and corresponds to a SAP of type 1 or 2 as defined in Annex I of [ISO BMFF].

A **sync AV2 sample** SHALL be self-sufficient for starting decoding at that point for all layers: every OBU referenced by the coded frame OBUs of the **sync AV2 sample** SHALL be available either by being carried in the **AV2CodecConfigurationBox** of the active **AV2SampleEntry** or by being present in the **sample**, as required by the **availability of HLS OBUs** in [AV2].

NOTE: In ISO BMFF terms, sync samples are signalled using the SyncSampleBox ('stss') in unfragmented tracks, or by setting `sample_is_non_sync_sample` equal to 0 in fragmented tracks. When every sample in a track is a sync sample, the SyncSampleBox is omitted in unfragmented tracks (per [ISO BMFF]), and in fragmented tracks this is expressed through the default sample flags of the MovieExtendsBox ('trex') or TrackFragmentHeaderBox ('tfhd') with no per-sample override.

§ 3.3.4. Layers

The following applies to each [coded extended layer unit](#) in an [AV2 sample](#) associated with a 'pixi' box:

- The channels in the [AV2 sample](#) are considered to be ordered as if the embedded layers were concatenated in increasing `m_layer_id` order.

EXAMPLE 2

An embedded layer with `m_layer_id` 0 and `CHROMA_FORMAT_420` contains the three first channels. Another embedded layer with `m_layer_id` 1 and `CHROMA_FORMAT_400` contains the fourth channel.

- If the 'pixi' box describes the Nth channel to be a color channel, the Nth channel in the [AV2 sample](#) SHALL be present and a color channel: if signalled, its associated `lcr_layer_type` SHALL be set to 0 (TEXTURE_LAYER).
- If the 'pixi' box describes the Nth channel to be an alpha channel, the Nth channel in the [AV2 sample](#) SHALL be present and an alpha channel: if signalled, its associated `lcr_layer_type` SHALL be set to 1 (AUX_LAYER) and `lcr_auxiliary_type` to 0 (ALPHA_AUX).
- If the 'pixi' box describes the Nth channel to be a depth channel, the Nth channel in the [AV2 sample](#) SHALL be present and a depth channel: if signalled, its associated `lcr_layer_type` SHALL be set to 1 (AUX_LAYER) and `lcr_auxiliary_type` to 1 (DEPTH_AUX).

EXAMPLE 3

A PixelInformationBox with `px_flags=1` and `channel_idc` set to 2, 3, 4, 1, 0, 0, 0, 5, 6 matches the following embedded layers: TEXTURE_LAYER/CHROMA_FORMAT_444, AUX_LAYER/SEGMENTATION_AUX/CHROMA_FORMAT_400, AUX_LAYER/GAIN_MAP_AUX/CHROMA_FORMAT_420, AUX_LAYER/ALPHA_AUX/CHROMA_FORMAT_400, AUX_LAYER/DEPTH_AUX/CHROMA_FORMAT_400.

EXAMPLE 4

A PixelInformationBox with `px_flags=1` and `channel_idc` set to 7, 2, 3, 4 matches the following embedded layers: TEXTURE_LAYER/CHROMA_FORMAT_400, TEXTURE_LAYER/CHROMA_FORMAT_444.

§ 3.3.5. AV2 sample reconstruction

A byte sequence containing a conformant [\[AV2\] Temporal Unit](#) using the [Length delimited bitstream format](#) specified in Annex B of [\[AV2\]](#) MAY be reconstructed from the OBUs of an [AV2 sample](#) by applying the following steps:

1. Write a [Temporal Delimiter OBU](#), preceded by its `leb128()` length prefix.
2. Write the OBUs carried in the [sample](#), in the same order as present in the [sample](#), each preceded by its `leb128()` length prefix as present in the [sample](#).

3. Insert each [OBU](#) carried in the [AV2CodecConfigurationBox](#) of the corresponding [AV2SampleEntry](#), preceded by its `leb128()` length prefix as present in the [AV2SampleEntry](#), between two `leb128()`-prefixed [OBUs](#) already present in the byte sequence, so that:
- the relative order of the [OBUs](#) carried in an [AV2CodecConfigurationBox](#) **SHALL** be kept in the reconstructed byte sequence,
 - the reconstructed byte sequence **SHALL** conform to the [ordering of OBUs in a temporal unit](#) as specified in [\[AV2\]](#), and
 - there **SHALL NOT** be an earlier insertion location for that [OBU](#) that would satisfy the above constraints.

NOTE: AV2-ISOBMFF reader implementations do not have to reconstruct the contiguous byte sequence containing a conformant [AV2 bitstream](#) or [Temporal Unit](#) in memory upon decoding, but they must have all the data to do so: all [OBUs](#), and enough information to deterministically reorder the [OBUs](#) if necessary.

§ 4. CMAF AV2 track format

TBD

§ 5. Common Encryption

TBD

§ 6. Codecs Parameter String

TBD

§ Index

§ Terms defined by this specification

[av02](#), in § 3.1

[AV2 sample](#), in § 3

[sync AV2 sample](#), in § 3.3.3

[AV2CodecConfigurationBox](#), in § 3.2.2

[AV2SampleEntry](#), in § 3.2.1

§ Terms defined by reference

[AV2] defines the following terms:

AV2 bitstream
availability of HLS OBUs
Bridge Frame OBU
Closed Loop Key OBU
Closed Random Access
coded extended layer unit
High-Level Syntax
Inter Frame
Intra frame
Key Frame
Layer Configuration Record OBU
Leading frame
Length delimited bitstream format
metadata group OBU syntax
Metadata OBU
metadata short OBU syntax
Multistream
OBU
obu_header_extension_flag
obu_mlayer_id
obu_tlayer_id
obu_type
obu_xlayer_id
Open Loop Key OBU
Open Random Access
ordering of OBUs in a temporal unit
sample
sample value
Sequence Header OBU
show_existing_frame
Singlestream
Switch Frame
Switch OBU
Temporal Delimiter OBU

Temporal Unit

[ISOBMFF] defines the following terms:

btrt
clli
colr
ctts
iso6
mdcv
nclx
pasp
pixi
sample
VisualSampleEntry

§ References

§ Normative References

[AV2]

AV2 Bitstream & Decoding Process Specification. Standard. URL: <https://av2.aomedia.org/v1.0.0/index.html>

[CMAF]

Information technology — Multimedia application format (MPEG-A) — Part 19: Common media application format (CMAF) for segmented media. Standard. URL: <https://www.iso.org/standard/71975.html>

[ISOBMFF]

Information technology — Coding of audio-visual objects — Part 12: ISO base media file format. Standard. URL: <https://www.iso.org/standard/83102.html>

[RFC2119]

Key words for use in RFCs to Indicate Requirement Levels. Standard. URL: <https://www.rfc-editor.org/info/rfc2119>

[RFC8174]

Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words. Standard. URL: <https://www.rfc-editor.org/info/rfc8174>